MAP (Mini Aplicación)
Serie de Fourier Truncada y FFT

1. Objetivo

Hacer uso de las series de Fourier en distintas aplicaciones de ingeniería, como encontrar la aproximación de la serie de Fourier truncada calculando la cantidad de términos con la integral cuadrada del error e incluyendo la eliminación de ruido en imágenes utilizando filtros pasivos. Se utilizarán las series de Fourier para descomponer señales y analizar cómo los filtros afectan su comportamiento. Adicionalmente, se explorará la intersección de estas técnicas clásicas con herramientas de *Inteligencia Artificial (IA)*. Se motivará al estudiante a utilizar modelos de IA generativa para la creación de señales personalizadas y a emplear bibliotecas de Python modernas para desarrollar aplicaciones más visuales e interactivas, conectando la teoría fundamental con las tecnologías más actuales en el campo de la ingeniería.

2. Marco Teórico

Las *Series de Fourier* describen señales periódicas como una combinación de señales armónicas. Uno de sus objetivo es mostrar que una función periódica puede descomponerse como *suma de funciones trigonométricas*, como senos y cosenos. Donde las frecuencias son múltiplos enteros de la frecuencia fundamental. La *serie de Fourier Trigonométrica (SFT)* está dada por:

$$f(t) = a_0 + \sum_{n=1}^{\infty} \left[a_n \cos(n\omega t) + b_n \sin(n\omega t)\right], \qquad \omega = \frac{2\pi}{T}$$

donde

$$a_0 = \frac{1}{T}\int_0^T f(t)\,dt, \qquad a_n = \frac{2}{T}\int_0^T f(t)\cos(n\omega t)\,dt, \qquad b_n = \frac{2}{T}\int_0^T f(t)\sin(n\omega t)\,dt.$$

Serie de Fourier truncada a $2N+1$ términos: al querer aproximar una función periódica $f(t)$ la cual es continua por partes y posee infinitos armónicos, tendremos que truncar la función hasta el armónico N. La forma de calcular la energía de la señal:

$$E_f = \int_0^T f^2(t)\,dt$$

Integral cuadrada del error (ICE): el cálculo de ICE se utiliza para encontrar el valor N que luego será utilizado para obtener una representación de la SFT.

$$ICE(N) = E_f - \left\{a_0^2 T + \frac{T}{2}\sum_{n=1}^{N}\left(a_n^2 + b_n^2\right)\right\}$$

$$ICE(N) \leq 0.02 E_f$$

Transformada de Fourier Discreta (DFT): es una técnica matemática utilizada para analizar las componentes de frecuencia de una señal discreta o una secuencia. Dada una secuencia $x[n]$ de longitud N, la DFT $\mathbf{X}[k]$ es calculada utilizando la siguiente fórmula:

$$\mathbf{X}[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-\frac{2\pi kni}{N}}$$

donde:

- $\mathbf{X}[k]$ es el k-ésimo elemento de la secuencia de salida de la DFT.
- $\mathbf{x}[n]$ es el n-ésimo elemento de la secuencia de entrada.
- i es la unidad imaginaria, $i^2 = -1$.
- k es el índice de la secuencia de salida, que va de 0 a $N-1$.
- N es el número total de muestras en la secuencia de entrada.

Transformada Rápida de Fourier (FFT) es un algoritmo eficiente para calcular la Transformada de Fourier Discreta. Reduce la complejidad computacional de la DFT de $O(N^2)$ a $O(N \log N)$ al aprovechar las propiedades de periodicidad de los exponenciales complejos. Uno de los algoritmos de FFT ampliamente utilizados es el *algoritmo FFT de Cooley-Tukey*, que divide de forma recursiva el cálculo de la DFT en subproblemas más pequeños. Los principales pasos del algoritmo FFT de Cooley-Tukey incluyen:

1. Dividir la secuencia de entrada en índices pares e impares para crear dos subproblemas más pequeños.
2. Calcular recursivamente la DFT de los subproblemas pares e impares.
3. Combinar los resultados de la DFT de los subproblemas pares e impares para calcular el resultado final de la DFT.

Al aplicar estos pasos de forma recursiva, el algoritmo FFT reduce significativamente la complejidad temporal del cálculo de la DFT.

3. Guía Práctica

En este proyecto, se dividirá en dos partes: *i*) series de Fourier para encontrar forma truncada de una serie a $2N+1$ términos, encontrar la energía de una función $f(t)$ y *ii*) aplicar los filtros generados por Fourier a imágenes.

Utilice *Google Colab* u otro entorno similar (consulte la Sección 5) para trabajar el MAP en un Notebook de Python, el cual usted creará para cada uno de los incisos siguientes. Emplee cualquier herramienta de *inteligencia artificial (IA)* (ver Sección 5) para modificar dicho Notebook. Cada consulta realizada a la IA debe ser *documentada* en el reporte con un *screenshot* del prompt y la respuesta obtenida, explicando cómo se utilizó. En el Notebook, desarrolle cada uno de los puntos que se detallan a continuación.



Parte 1 – Series de Fourier

1. Implemente una interfaz gráfica que cumpla con los siguientes requerimientos.

   - Permitir la selección de función a utilizar. Considere las dos funciones:

$$i)\quad f_1(t)=\begin{cases}0, & -2<t<-1,\\ 1, & -1<t<1,\\ 0, & 1<t<2.\end{cases}\qquad ii)\quad f_2(t)=\begin{cases}0, & -2<t<-1,\\ t+2, & -1<t<1,\\ 0, & 1<t<2.\end{cases}$$

   - Los datos desplegados por su interfaz al usuario deben ser:
     - Período de la función.
     - Coeficientes de la serie de Fourier para la función.
     - Gráfica aproximada de la serie de Fourier trigonométrica truncada a $2N+1$ términos.
     - Valor de N.
     - Ecuación de la serie de Fourier trigonométrica.

2. Documente paso a paso, es decir colocando todos los datos generados por su interfaz, para los valores $N = 3, 5, 10$ para cada función.

3. Con base a los resultados anteriores, responda:

   a) ¿En cuál de las dos funciones converge más rápido la serie de Fourier Trigonométrica?

   b) ¿Es útil el cálculo de ICE para ambas funciones?

   c) ¿Con cuántas iteraciones podemos obtener una mejor aproximación para las funciones dadas?

4. Utilice un modelo de lenguaje grande (LLM) como Gemini, ChatGPT o similar, para que le ayude a *generar el código en Python* de una función periódica original y no trivial. Puede darle instrucciones (prompts) como:

   > *"Genera una función periódica en Python que modele el sonido de un instrumento musical exótico. La función debe ser una combinación de al menos tres sinusoides con diferentes amplitudes y frecuencias, y debe incluir alguna discontinuidad o característica interesante. La función debe estar definida en un periodo claro."*

5. Una vez generada la función, utilice la interfaz que creó para analizarla. Documente la serie de Fourier truncada para $N = 15$ y analice si la aproximación es adecuada.

6. Grafique tanto la señal original generada por IA como su aproximación de Fourier. Use bibliotecas como 'matplotlib' o 'seaborn' en Python para crear una visualización de alta calidad. ¡Experimente con colores y estilos para hacerla más interesante!



Parte 2 – FFT y Filtros de Imagen Avanzados

A continuación se presenta el algoritmo de FFT, el cual deberá utilizar para los incisos siguientes.

Cooley-Tukey Fast Fourier Transform (FFT) Algorithm

```
function FFT(x, N)
    if N = 1 then
        return x
    end if
    x_even ← FFT([x_0, x_2, ..., x_{N-2}], N/2)      ▷ Secuencia de índices pares
    x_odd ← FFT([x_1, x_3, ..., x_{N-1}], N/2)       ▷ Secuencia de índices impares
    for k ← 0 to N/2 − 1 do
        T ← x_odd[k] × e^{−2πki/N}                   ▷ i es la unidad imaginaria
        X[k] ← x_even[k] + T
        X[k + N/2] ← x_even[k] − T
    end for
    return X
end function
```

Considere las imágenes adjuntas en el proyecto. Si usted desea utilizar sus propias imágenes puede hacerlo. En base algoritmo anterior, realice los incisos que se detallan a continuación.

1. Aplique el algoritmo a las imágenes proporcionadas utilizando un *filtro pasa bajas* con las siguientes frecuencias de corte:

   a) $f_{\text{cut off}} = 0.22$,

   b) $f_{\text{cut off}} = 0.15$,

   c) $f_{\text{cut off}} = 0.96$,

   ¿Cuál es la diferencia entre las imágenes originales y las generadas? Justifique su respuesta analizando el filtro pasa bajas.

2. Aplique el algoritmo a las imágenes proporcionadas utilizando un *filtro pasa altas* con las siguientes frecuencias de corte:

   a) $f_{\text{cut off}} = 0.23$,

   b) $f_{\text{cut off}} = 0.11$,

   c) $f_{\text{cut off}} = 0.91$,

   ¿Cuál es la diferencia entre las imágenes originales y las generadas? Justifique su respuesta analizando el filtro pasa altas.

3. Modifique su implementación del algoritmo o la implementación utilizada para que pueda realizar *Blur Detection*. Luego, aplique su implementación con Blur Detection a las imágenes proporcionadas.

4. Investigue sobre filtros de imagen basados en redes neuronales convolucionales (CNN). No es necesario que entrene una red desde cero. Busque un modelo pre-entrenado sencillo para una tarea como **eliminación de ruido (denoising)** y **mejora de la resolución (super-resolution)**. Bibliotecas como 'TensorFlow/Keras' o 'PyTorch' tienen tutoriales y modelos disponibles.

   a) Agregue su implementación y utilícela para las imágenes adjuntas en el proyecto.

   b) Tome una de las imágenes originales y agréguele ruido artificialmente (puede usar la biblioteca 'scikit-image' o 'OpenCV').

   c) Aplique los filtros de FFT implementados para intentar limpiar la imagen.

   d) Aplique el filtro basado en IA a la misma imagen con ruido.

5. ¿Cuál filtro funcionó mejor? Justifique su respuesta mostrando las imágenes resultantes y explicando brevemente cómo la IA puede superar a los filtros clásicos en ciertas tareas al "aprender" las características de una imagen limpia.

4. Instrucciones Generales

1. Este proyecto se realizará en grupos de un máximo de *tres* integrantes.

2. Deben enviar un correo o un mensaje en Telegram a su asistente de cátedra *antes del miércoles 21 de Octubre de 2026* para la creación de los grupos de trabajo en el GES. Dicho correo o mensaje debe incluir el nombre y carné de cada uno de los integrantes del grupo.

3. Su reporte deberá realizarlo usando LaTeX.

4. Deberá realizar un pequeño reporte que incluya la respuesta a cada uno de los inciso que se detallan en la sección 3. Recuerde que respuesta no justificada no recibirá calificación, por ello debe dejar constancia de todo razonamiento y procedimiento realizado.

5. **Fecha de Entrega: Domingo 8 de Noviembre de 2026**. Su solución debe subirla en un archivo ZIP enviado por GES y debe contener:

   - Archivo editable de LaTeX (.tex) de su reporte.
   - Imágenes utilizadas en su archivo de LaTeX.
   - Su reporte en formato PDF generado por su archivo de LaTeX.
   - El Notebook de Python utilizado en ambas partes.

   de lo contrario su trabajo *no* será tomado en cuenta.

5. Referencias

Acerca de Series de Fourier.

a) Notas de clase, Portal de Matemática VI en GES.

b) Dyke, P.P.G.: *An Introduction to Laplace Transforms and Fourier Series*, Springer Undergraduate Mathematics Series. Springer, 2a. edición, 2014.

c) Zill, D., W. Wright y M. Cullen: *Matemáticas Avanzadas para Ingeniería*, McGraw-Hill, 4a. edición, 2012.

Herramientas de Graficación.

a) Página oficial de SageMath - http://www.sagemath.org/

b) Página oficial de Cocalc - https://cocalc.com/

Acerca de LaTeX.

a) Página oficial de MikTex - https://miktex.org/

b) Página oficial de TeXstudio - https://www.texstudio.org/

c) Página oficial de TeXmaker - https://www.xm1math.net/texmaker/

d) Página oficial de Overleaf - https://www.overleaf.com/

Inteligencia Artificial (IA) y Aplicaciones en Matemática

a) OpenAI. *Documentación de la API de ChatGPT*. 2023. https://platform.openai.com/docs

b) Google Colab. *Integración con modelos de IA*. https://colab.research.google.com/

c) Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press, 2016. https://www.deeplearningbook.org/

d) Wolfram, S. *AI for Mathematics*. Wolfram Research, 2023. https://www.wolfram.com/

e) Chollet, F. *Deep Learning con Python*. Manning Publications, 2a edición, 2021.